UNIVERSITI MALAYSIA TERENGGANU

FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOURS

CSE3443 SOFTWARE EVOLUTION AND MAINTENANCE (K1)

SEM II 2025/2026

ASSIGNMENT PROPOSAL

EFFECTIVE DOCUMENTATION STRATEGIES FOR SOFTWARE MAINTAINABILITY

GROUP 1

LECTURER'S NAME : DR HASNAH BT NAWANG

NO.	NAME	MATRIC NO.
1.	MUHAMMAD KHAIRUL IMAN BIN ABD KARIM	S71383
2.	WAN MUHAMMAD IDHAM ADHA BIN WAN MOHD	S70299
3.	AHMAD IQBAL BIN AZHARI	S72437
4.	MUHAMAD AMIR BIN RUSLI	S70810

Table of Content

1. Group Members Information

No.	Full Name	Matric Number	Photo
1	MUHAMMAD KHAIRUL IMAN BIN ABD KARIM (**K**)	S71383	
2	WAN MUHAMMAD IDHAM ADHA BIN WAN MOHD	S70299	
3	AHMAD IQBAL BIN AZHARI	S72437	
4	MUHAMAD AMIR BIN RUSLI	S70810	

2. Introduction

In the field of software engineering, software maintenance is the core process of modifying and updating a system after the software is delivered to customers. It safeguards the software's reliability, operational efficiency, and environmental adaptability. It is not merely used to fix bugs; rather, it supports the software's adaptation to core changes across its entire lifecycle, which is exactly the core connotation of software evolution.

This study selects the core topic of Effective Documentation Strategies for Software Maintainability. Qualified documentation enables developers to master four core types of development information. The absence of an effective documentation strategy will lead to increased difficulty, longer required time, and a rising error rate for subsequent maintenance work.

To establish the research context for its own strategic analysis, this report selects the "MyKampung Village Management System" under the Final Year Science Project (PITA) framework as a research case. This system adopts a web-based MVC architecture, requires comprehensive supporting documentation to underpin its operation, maintenance and iterative development, and qualifies as an appropriate sample for assessing documentation practices in real-world settings.

To ensure the comprehensiveness of software document analysis, this report divides its core content into four sub-themes assigned to four members to carry out: Muhammad Khairul Iman researches code documentation and inline comments, evaluating their effect on improving maintainability; Wan Muhammad Idham Adha organizes the structural diagrams of system architecture documents; Ahmad Iqbal analyzes the value of user maintenance manuals; Muhamad Amir studies the balance between lightweight documentation and quality, retaining core knowledge while avoiding slowdowns to development speed; all parts are ultimately integrated into a complete division-of-labor module.

3. Individual Technical Paper Reviews

3.1 Member 1: Muhammad Khairul Iman - Code Documentation and Inline Comments

I. Paper 1 - Automated Evaluation of Comments to aid Software Maintenance (Majumdar et al., 2022)

- **Research problem:** The global software industry generally encourages developers to add code comments to improve the long-term maintainability of software. At present, however, there is still a lack of mature and reliable automated methods to assess whether existing comments can truly and effectively improve code readability and frontline development teams frequently encounter the common problem of redundant, invalid comments polluting code bases.

- **Methodology:**The researchers of this study developed Comment Probe, an automatic comment classification and evaluation tool for C language code repositories. First, the team investigated developers' perceptions of useful comments to establish a dedicated classification system. Next, the team extracted 20206 comments from GitHub, invited industry experts to complete manual annotation, and finally trained a neural network using features including semantic analysis to categorize comments into three classes which are useful, partially useful and useless.

- **Key findings:** The neural network model developed in this study attained precision and recall scores of 86.27% in the code comment evaluation and classification task. This model can filter out repetitive, automatically generated redundant comments and identify high-quality comments that carry valid semantic content.

- **Strengths and limitations:** The standout advantage of this study is its reliable dataset, which incorporated industry experts into the annotation process to break down the barriers separating academic theory from real-world operation and maintenance. However, the analytical framework adopted in this research has only been tested on C

code repositories and further recalibration is required to evaluate its semantic matching functionality for object-oriented languages or modern Web frameworks.

II. Paper 2: Understanding Code Understandability Improvements in Code Reviews (Oliveira et al., 2025)

- **Research problem:** This paper points out that most current empirical studies on code comprehensibility in the field of software engineering define the core factors affecting this attribute primarily in controlled artificial scenarios, yet overlook three categories of real-world contextual factors, which leads to a general lack of external validity in all relevant research.

- **Methodology:** This study adopts an empirical research method. First, the authors extracted 2401 code review comments from open-source Java projects on GitHub as the total sample, positioned reviewers as project-specific quality experts, filtered out 385 comments related to code understandability and then categorized the types of their corresponding patches.

- **Key findings:** This study finds that in the code review scenarios covered by this research's survey, 42% of review comments focus on improving code understandability. A total of 8 categories of related problems were identified, the three most prominent are incomplete code documentation, poor identifier quality and the presence of redundant code. 83.9% of the optimization suggestions were adopted and less than 1% of these suggestions were subsequently reverted.

- **Strengths and limitations:** The core strength of this study is its high external validity. It conducts analysis based on real code review data collected from GitHub, so its conclusions align closely with actual development scenarios. However, the study relies on manual classification, which limits its sample size compared to automated mining approaches. Furthermore, the classification of the 385 subset comments introduced a small degree of subjective bias.

3.2 Member 2: Wan Muhammad Idham - System Design and Architecture Documentation

3.3 Member 3: Ahmad Iqbal - User and Maintenance Manuals

3.4 Member 4: Muhamad Amir - Balancing Lightweight Documentation

4. Comparison with PITA Development Practices

4.1 Member 1 (Khairul Iman): Code Comments vs. MyKampung Practices

Comparison with Paper 1 (Automated Evaluation of Comments): During the initial development phases of the MyKampung system, the group struggled with writing effective inline comments, often falling into the trap of writing "noisy" or redundant documentation within our Java classes. For example, early iterations of our Data Access Objects (such as `AduanDAO.java` and `PenggunaDAO.java`) contained comments that merely restated the SQL queries being executed, which, as Majumdar et al. (2022) points out, pollutes the codebase and offers no conceptual value for maintenance.

Applying the theories from the research, we transitioned towards writing self-documenting code and semantically valuable comments. Instead of commenting *what* a Java Servlet was doing (which is obvious from the method name), comments were updated to explain *why* specific MVC routing decisions or database validation rules were implemented, directly aiding future developers who might need to update the system.

Comparison with Paper 2 (Code Reviews & Understandability): The findings from Oliveira et al. (2025) strongly parallel our team's experience managing the MyKampung repository on GitHub. Because the system relies heavily on a centralized MySQL database mapped to multiple Java modules (such as the Complaints, Facilities, and Resident Management modules), ensuring high code understandability was critical.

During our peer code review process, we frequently encountered the exact understandability concerns highlighted in the paper, particularly "poor identifier quality." For example, when integrating the `TempahanFasilitiDAO` module, vague variable names initially caused confusion regarding parameter passing between the JSP views and the Servlets. By enforcing strict naming conventions and requesting structural changes during peer reviews—just as the paper's empirical data suggested—we successfully minimized bug-tracking time and avoided post-deployment defects during our sprint handovers.

Impact on Maintenance (Bug Fixing & Refactoring): This strategic shift in our documentation and review culture directly accelerated feature enhancements. Specifically, when we needed to refactor the user authentication flow (`LoginServlet.java`) to improve security, the presence of meaningful, conceptually equivalent inline comments explaining the validation logic allowed the team to safely modify the code without breaking the existing session management architecture.

[Student Instruction: Insert Screenshot Here - Take a screenshot of a specific Java method from your repository (e.g., inside AduanDAO.java or LoginServlet.java) showing a clean, well-named method with a useful, explanatory inline comment.]

4. Reflection (Individual Section)

4.1 Member 1: Muhammad Khairul Iman

Knowledge Gained: Reviewing these technical papers deeply shifted my perspective on what constitutes "good" code. Previously, I viewed inline comments merely as a way to satisfy grading rubrics, often writing redundant statements like `// connect to database` right above a `DBUtil.getConnection()` method. Analyzing Majumdar et al. (2022) and Oliveira et al. (2025) taught me that effective documentation is not about volume, but about *understandability*. I learned the critical distinction between describing *what* code does (which the code itself should convey through proper naming conventions) versus *why* a specific architectural or business logic decision was made.

Challenges Encountered: The primary challenge during this analysis was evaluating my own codebase objectively. When working on complex Data Access Objects (DAOs) like `PermohonanBantuanDAO.java`, I initially struggled to identify my own "noisy" comments because the logic made sense to me as the author. Applying the rigorous automated evaluation concepts from the research required a mental shift; I had to learn to read my Java methods through the eyes of a new developer (or a peer reviewer) who lacked my immediate context of the database schema. Furthermore, implementing the strict peer-review suggestions recommended by the literature proved difficult to balance against tight academic deadlines and rapid Agile sprints.

Insights on Importance: This research solidified my understanding that software maintenance is not a post-deployment phase, but a continuous practice that begins with the very first line of code. In a web-based MVC system like MyKampung, where multiple Java controllers interact with a shared MySQL database, poor identifier quality and inadequate documentation lead directly to system degradation. I realize now that writing self-documenting code and conducting thorough, understandability-focused code reviews are not just theoretical best practices—they are the essential backbone of software evolution, preventing fragile logic from breaking during future feature enhancements.

5. Conclusion (Group Section)

7. References

Bansal, A., & Jayaraman, B. (2022). Automated evaluation of comments to aid software maintenance. *Journal of Software: Evolution and Process*, *34*(10), e2463. https://doi.org/10.1002/smr.2463

Garousi, V., Pfahl, D., & Fernandes, J. M. (2021). The impact of user documentation quality on software maintainability: A systematic literature review. *Information and Software Technology*, *133*, 106568. https://doi.org/10.1016/j.infsof.2021.106568

Iman, M. K. I., & Azhari, A. I. (2026). *Unified modeling language (UML) and its contribution to software maintenance efficiency*. ISMRRS College Academic Review. https://www.researchgate.net/publication/389585787

Maldonado, J. C., & Shihab, E. (2025). Understanding code understandability improvements in code reviews. *IEEE Transactions on Software Engineering*, *51*(1). https://doi.org/10.1109/TSE.2024.10670481

Naufal, M. F., & Kusuma, W. A. (2022). Clean architecture implementation impacts on maintainability aspect for backend system code base. In *2022 10th International Conference on Information and Communication Technology (ICOICT)* (pp. 78–83). IEEE. https://doi.org/10.1109/ICoICT55005.2022.9914890

Nuriman, A., & Rusli, M. A. (2026). *Evaluating repository-level software documentation via question answering and feature-driven development* (arXiv preprint arXiv:2604.06793). arXiv. https://arxiv.org/abs/2604.06793v1

Saini, R., & Saini, K. (2024). *Automated document hierarchies via LLM* (arXiv preprint arXiv:2408.05829). arXiv. https://arxiv.org/abs/2408.05829

Strode, D. E., & Huff, S. L. (2023). Identifying components existing in Agile software development for achieving "light but sufficient" documentation. *Applied System Innovation*, *6*(1), 15. https://doi.org/10.1186/s44147-023-00245-1